Lab Assignment No. 01 - Programming for AI

University: Riphah International University Sahiwal
Semester / Section: FA25 BS AI - B
Instructor: Muhammad Usama Arshad

Q1) Student Result Management System

```
def get_number(prompt, low, high, cast=int):
    """Keep asking until the user enters a valid number between low and high."""
    while True:
        try:
            value = cast(input(prompt))
            if low <= value <= high:
                return value
            print(f"  Please enter a value between {low} and {high}.")
        except ValueError:
            print("  Invalid input. Please enter a number.")


def calculate_result(student):
    """Returns total marks, percentage and grade for one student."""
    marks = student["marks"]
    total = sum(marks.values())
    percentage = total / (len(marks) * 100) * 100   # each subject is out of 100

    if percentage >= 80:
        grade = "A"
    elif percentage >= 70:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
    elif percentage >= 50:
        grade = "D"
    else:
        grade = "F"

    return total, percentage, grade

subjects = ["Python", "Artificial Intelligence", "Mathematics"]
students = []   # list that stores all student dictionaries

for i in range(1, 6):
    print(f"\n--- Student {i} ---")
    name = input("Enter name: ").strip()
    roll_no = input("Enter roll number: ").strip()

    marks = {}
    for subject in subjects:
        marks[subject] = get_number(f"Enter marks in {subject} (0-100): ", 0, 100)

    student = {"name": name, "roll_no": roll_no, "marks": marks}
    total, percentage, grade = calculate_result(student)
    student["total"] = total
    student["percentage"] = percentage
    student["grade"] = grade
    students.append(student)

# Display results of all students + find topper + save to results.txt
lines = []
lines.append("=" * 60)
lines.append("STUDENT RESULTS")
lines.append("=" * 60)

for s in students:
    lines.append(f"Name        : {s['name']}")
    lines.append(f"Roll No     : {s['roll_no']}")
    for subject, m in s["marks"].items():
        lines.append(f"  {subject:<25}: {m}")
    lines.append(f"Total Marks : {s['total']} / {len(subjects) * 100}")
```

```
    lines.append(f"Percentage  : {s['percentage']:.2f}%")
    lines.append(f"Grade       : {s['grade']}")
    lines.append("-" * 60)

topper = max(students, key=lambda s: s["percentage"])
lines.append("HIGHEST PERCENTAGE")
lines.append(f"{topper['name']} (Roll No: {topper['roll_no']}) - "
             f"{topper['percentage']:.2f}% - Grade {topper['grade']}")
lines.append("=" * 60)

report = "\n".join(lines)
print(report)

with open("results.txt", "w", encoding="utf-8") as f:
    f.write(report)
print("\nResults saved in results.txt")
```

Q2) Sentence / Word Analysis

```
import string


def clean_words(sentence):
    """Split sentence into words and remove punctuation around them."""
    words = [w.strip(string.punctuation) for w in sentence.split()]
    return [w for w in words if w]          # drop empty strings


def count_vowels(sentence):
    return sum(1 for ch in sentence.lower() if ch in "aeiou")


def analyze_sentence(sentence):
    words = clean_words(sentence)
    if not words:
        return None

    return {
        "total_words": len(words),
        "total_chars": len(sentence),
        "chars_no_spaces": len(sentence.replace(" ", "")),
        "longest": max(words, key=len),
        "shortest": min(words, key=len),
        "vowels": count_vowels(sentence),
        "unique_words": set(w.lower() for w in words),
    }


sentence = input("Enter a sentence: ")
result = analyze_sentence(sentence)

if result is None:
    print("No words found in the sentence.")
else:
    sorted_words = sorted(result["unique_words"])
    output = [
        "WORD ANALYSIS",
        "=" * 40,
        f"Sentence                : {sentence}",
        f"Total words             : {result['total_words']}",
        f"Total characters        : {result['total_chars']} (with spaces)",
        f"Characters (no spaces)  : {result['chars_no_spaces']}",
        f"Longest word            : {result['longest']}",
        f"Shortest word           : {result['shortest']}",
        f"Number of vowels        : {result['vowels']}",
        f"Unique words (set)      : {result['unique_words']}",
        f"Alphabetical order      : {sorted_words}",
    ]
    text = "\n".join(output)
    print(text)

    with open("word_analysis.txt", "w", encoding="utf-8") as f:
        f.write(text)
    print("\nAnalysis saved in word_analysis.txt")
```

Q3) Shopping Program

```
def calculate_bill(products):
    """Returns subtotal, discount amount and final bill."""
    subtotal = sum(p["total"] for p in products.values())
    discount = subtotal * 0.10 if subtotal > 10000 else 0
    return subtotal, discount, subtotal - discount


products = {}   # dictionary: product name -> info

for i in range(1, 6):
    print(f"\n--- Product {i} ---")
    name = input("Product name: ").strip()
    price = get_number("Price (Rs.): ", 0, float("inf"), float)
    quantity = get_number("Quantity: ", 1, 100000, int)
    products[name] = {"price": price, "quantity": quantity, "total": price * quantity}

subtotal, discount, final_bill = calculate_bill(products)
expensive_name = max(products, key=lambda n: products[n]["price"])

lines = ["=" * 55, "SHOPPING BILL", "=" * 55,
         f"{'Product':<18}{'Price':>10}{'Qty':>6}{'Total':>12}", "-" * 55]
for name, p in products.items():
    lines.append(f"{name:<18}{p['price']:>10.2f}{p['quantity']:>6}{p['total']:>12.2f}")
lines += ["-" * 55,
          f"{'Total Bill':<34}Rs. {subtotal:>10.2f}"]
if discount > 0:
    lines.append(f"{'Discount (10%)':<34}Rs. {discount:>10.2f}")
else:
    lines.append("No discount (bill is not greater than Rs. 10,000)")
lines += [f"{'FINAL BILL':<34}Rs. {final_bill:>10.2f}",
          "=" * 55,
          f"Most expensive product: {expensive_name} (Rs. {products[expensive_name]['price']:.2f})"]

bill_text = "\n".join(lines)
print(bill_text)

with open("bill.txt", "w", encoding="utf-8") as f:
    f.write(bill_text)
print("\nBill saved in bill.txt")
```

Q4) Calculator

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed"
    return a / b

def modulus(a, b):
    if b == 0:
        return "Error: Modulus by zero is not allowed"
    return a % b


def read_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("  Invalid number, try again.")


while True:
    num1 = read_number("\nEnter first number : ")
    num2 = read_number("Enter second number: ")
    op = input("Select operation (+, -, *, /, %): ").strip()

    if op == "+":
        result = add(num1, num2)
    elif op == "-":
        result = subtract(num1, num2)
    elif op == "*":
        result = multiply(num1, num2)
    elif op == "/":
        result = divide(num1, num2)
    elif op == "%":
        result = modulus(num1, num2)
    else:
        result = "Error: Invalid operation selected"

    print(f"Result: {result}")

    again = input("Do you want to perform another calculation? (yes/no): ").strip().lower()
    if again not in ("yes", "y"):
        print("Calculator closed. Goodbye!")
        break
```

Q5) Restaurant Ordering System

```
menu = {
    1: {"item": "Burger", "price": 500},
    2: {"item": "Pizza",  "price": 800},
    3: {"item": "Fries",  "price": 250},
    4: {"item": "Drink",  "price": 150},
}


def show_menu():
    print("\n========== MENU ==========")
    for number, info in menu.items():
        print(f"{number}. {info['item']:<10} Rs. {info['price']}")
    print("==========================")


def calculate_order_bill(order):
    return sum(entry["total"] for entry in order)


order = []   # list of ordered items

while True:
    show_menu()
    choice = get_number("Select item number: ", 1, len(menu), int)
    quantity = get_number("Enter quantity: ", 1, 1000, int)

    item = menu[choice]
    order.append({
        "item": item["item"],
        "price": item["price"],
        "quantity": quantity,
        "total": item["price"] * quantity,
    })
    print(f"Added {quantity} x {item['item']}")

    more = input("Do you want to order more items? (yes/no): ").strip().lower()
    if more not in ("yes", "y"):
        break

final_bill = calculate_order_bill(order)

lines = ["=" * 50, "RESTAURANT BILL", "=" * 50,
         f"{'Item':<12}{'Price':>8}{'Qty':>6}{'Total':>12}", "-" * 50]
for entry in order:
    lines.append(f"{entry['item']:<12}{entry['price']:>8}{entry['quantity']:>6}{entry['total']:>12}")
lines += ["-" * 50, f"{'FINAL BILL':<26}Rs. {final_bill:>10}", "=" * 50, ""]

bill_text = "\n".join(lines)
print("\n" + bill_text)

with open("orders.txt", "a", encoding="utf-8") as f:   # "a" keeps previous orders
    f.write(bill_text + "\n")
print("Bill saved in orders.txt")
```